



life.augmented



STM32WL55

Dual core project creation, coding and debug

T.O.M.A.S. Team

Goals of the session

Using STM32CubeIDE we will demonstrate how to:

- start dual-core project for STM32WL55J device
- configure peripherals and assign them to specific core
- prepare dual-core code and build the project
- debug dual-core code using STLink

Prerequisites

- NUCLEO-WL55JC2 board
- Micro USB cable
- PC with preinstalled the following software:
 - STM32CubeIDE (in version 1.5.0 or newer)
 - STM32WL Cube library (in version 1.0.0 or newer)

Application details – configuration part

- Start dual-core project for STM32WL55J device
- Assign selected peripherals to specific core
(refer to the table on the right)
- Enable interrupts on EXTI lines where B1-B3 buttons are connected
- Enable internal pull-ups on B1-B3 buttons lines
- Keep default clock settings (4 MHz from MSI) for both cores
- Generate the code for STM32CubeIDE

	pins	CM0PLUS	CM4
B1 buton	PA0	+	
B2 buton	PA1		+
B3 buton	PC6	+	
LED1	PB15		+
LED2	PB9	+	
LED3	PB11		+

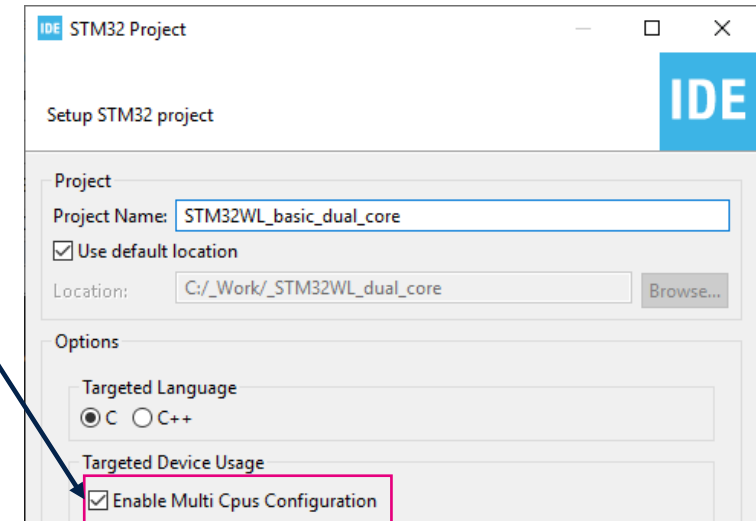
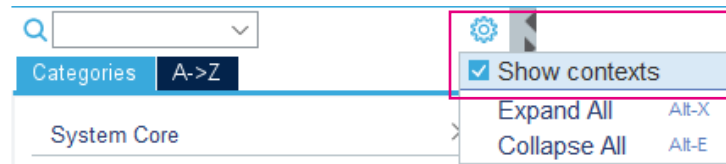
STM32WL55 dual core project creation



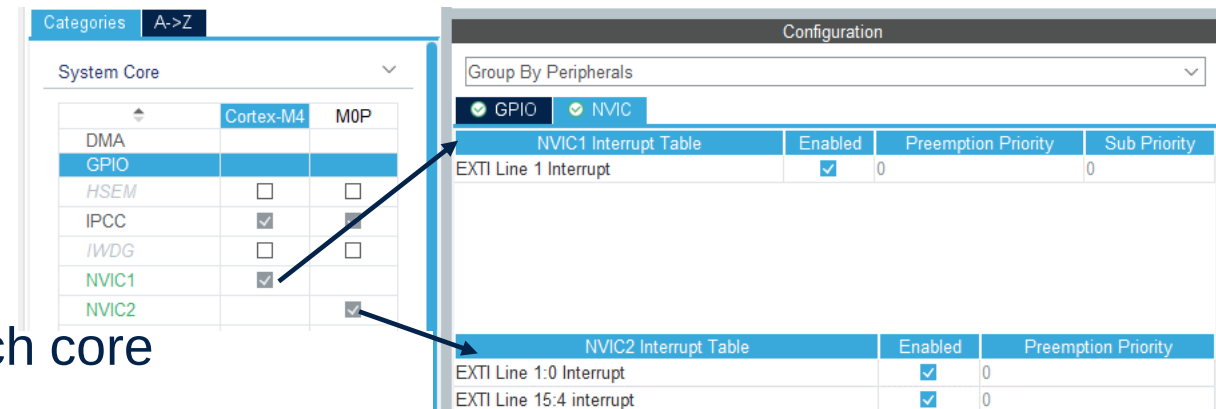
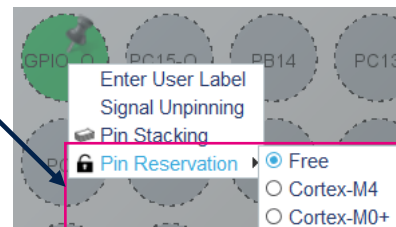
Application details – configuration hints

Some project preparation hints:

- Enable “Multi CPUs” configuration during project creation phase
- Within peripherals configuration enable „Show contexts”



- You can assign the pin to any core by clicking right mouse button and selecting „Pin Reservation”.



- There are two separate NVICs – one per each core

Application details – code generation

During project creation STM32CubeMX and STM32CubeIDE

1. Generate code for both cores (marked CM0PLUS and CM4)

- By default, only CM4 core is active after RESET.

To activate CM0PLUS core dedicated command needs to be executed:

```
HAL_PWREx_ReleaseCore(PWR_CORE_CPU2);
```

2. Create two linker files with the following memory usage settings:

- ARM CortexM4
 - FLASH starts at 0x0800 0000 (128k size), RAM starts at 0x2000 0000 (32k size)
- ARM CortexM0+
 - FLASH starts at 0x0802 0000 (128k size), RAM starts at 0x2000 8000 (32k size)

Dual core project modification



Application details – coding part

- Once per second we will increment *var_CM4* and *var_CM0P* variables in main while() loop for both cores
- The rest of user code will be stored in ext. interrupt callbacks:
 - B1 button press should toggle LED2
 - B2 button press should toggle LED1 and LED3
 - B3 button press should toggle LED2
- Definition of the callback prototype (w/ „weak” attribute) can be found in **stm32g0xx_hal_gpio.c** file (to be tracked from EXTI IRQ handler in **stm32g0xx_it.c** file):

	pins	CM0PLUS	CM4
B1 buton	PA0	+	
B2 buton	PA1		+
B3 buton	PC6	+	
LED1	PB15		+
LED2	PB9	+	
LED3	PB11		+

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin);
```

Application details – coding part

- Once per second we will increment *var_CM4* and *var_CM0P* variables in main while() loop for both cores

```
/* USER CODE BEGIN PV */
uint16_t var_CM0P=0;

....

/* USER CODE BEGIN WHILE */
while (1)
{
    var_CM0P++;
    HAL_Delay(1000);
/* USER CODE END WHILE */
```

main.c file within ARM CortexM0+ code

```
/* USER CODE BEGIN PV */
uint16_t var_CM4=0;

....

/* USER CODE BEGIN WHILE */
while (1)
{
    var_CM4++;
    HAL_Delay(1000);
/* USER CODE END WHILE */
```

main.c file within ARM CortexM4 code

Application details – coding part

- We need to implement external interrupts callbacks for both cores within **main.c** files (i.e. in USER CODE 4 section)

```
/* USER CODE BEGIN 4 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if(GPIO_PIN_0 == GPIO_Pin)
    {
        HAL_GPIO_TogglePin(LED2_GPIO_Port, LED2_Pin);
    }
    else if(GPIO_PIN_6 == GPIO_Pin)
    {
        HAL_GPIO_TogglePin(LED2_GPIO_Port, LED2_Pin);
    }
}
/* USER CODE END 4 */
```

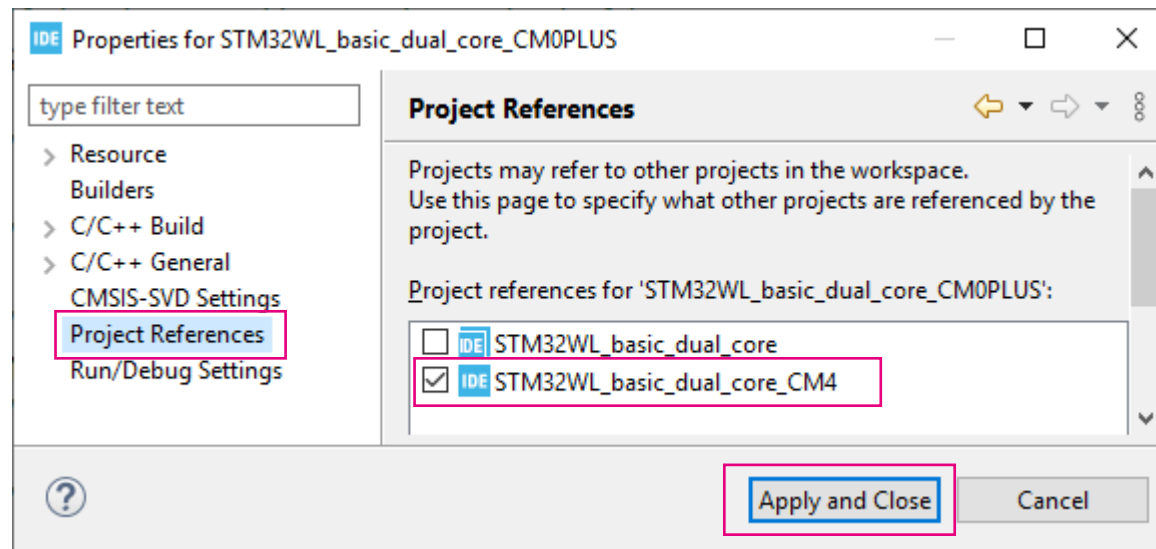
main.c file within CortexM0+ code

```
/* USER CODE BEGIN 4 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if(GPIO_PIN_1 == GPIO_Pin)
    {
        HAL_GPIO_TogglePin(LED1_GPIO_Port, LED1_Pin);
        HAL_GPIO_TogglePin(LED3_GPIO_Port, LED3_Pin);
    }
}
/* USER CODE END 4 */
```

main.c file within CortexM4 code

Application build configuration

- There are two independent projects for both cores; one of them can be configured for automatic build of both projects in the same order (suggested is to build CM4 first and then CM0PLUS).
- To perform such configuration, click right mouse button on CM0PLUS project name, select Properties → Project References and select CM4 project



- Now clicking on build button will trigger build of CM4 project first and then CM0PLUS one, whatever active project will be.

Dual core project debug configuration



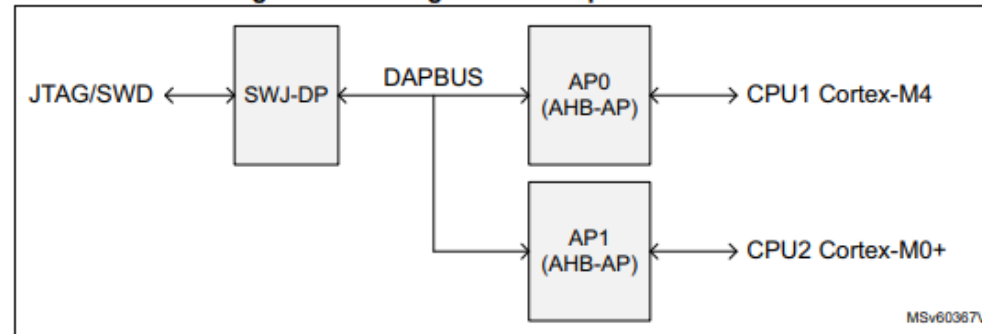
STM32WL55 debug ports extract from Reference Manual (RM0453)

- Within dual core STM32WL55 MCU there are 2 debug ports:
 - Port 0 – dedicated to CortexM4 core
 - Port 1 – dedicated to CortexM0+ core

RM0453

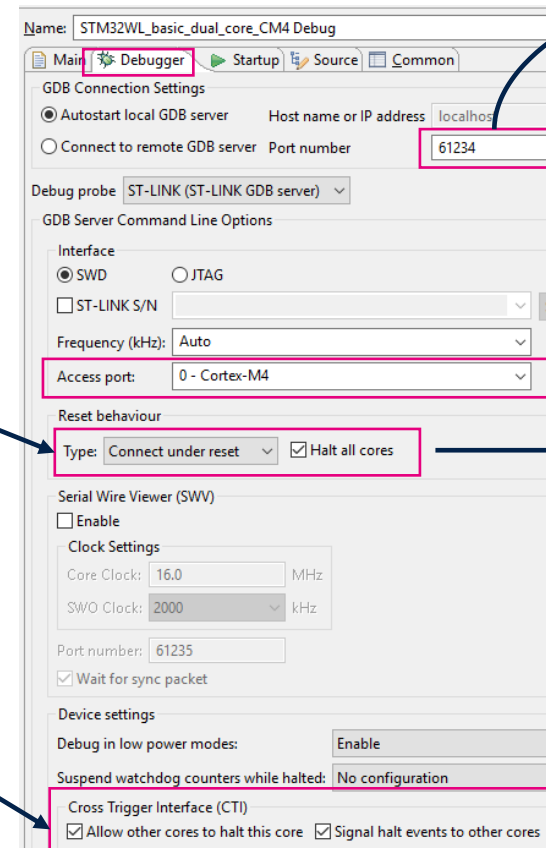
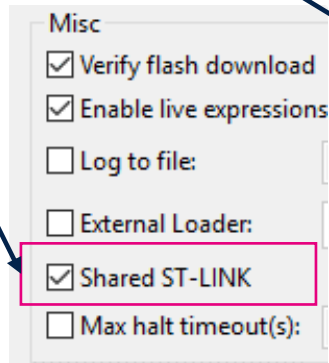
Debug support (DBG)

Figure 385. Debug and access port connections

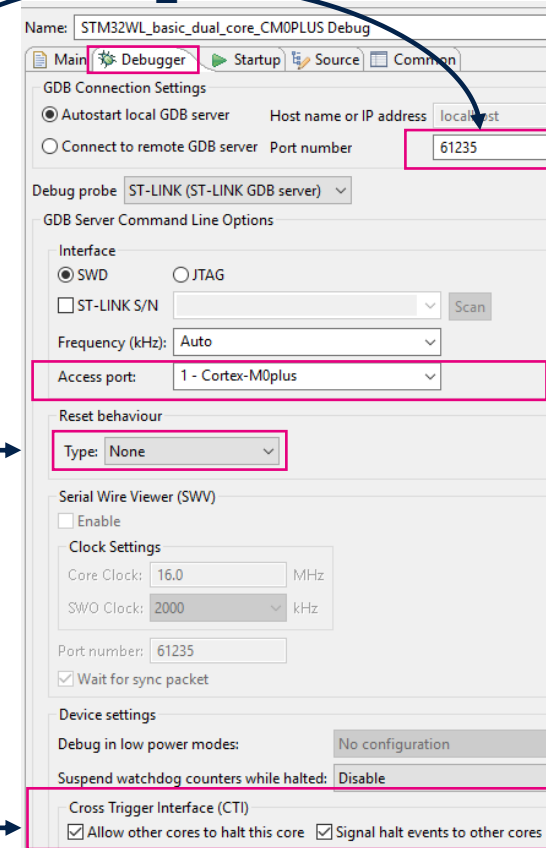


STM32WL55 debug ports configuration of debug session in STM32CubeIDE

- We need to specify correctly GDB Port number :
 - For CortexM4 core → Port number **61234** (default)
 - For CortexM0+ core → Port number = port selected for CortexM4 + **1 (or higher)**
- Select access port:
 - „0 – Cortex-M4” for CortexM4 part
 - „1 – CortexM0plus” for CortexM0+ part
- Select reset behavior only for CorexM4
 - both options within CTI part
 - „Shared ST-LINK”



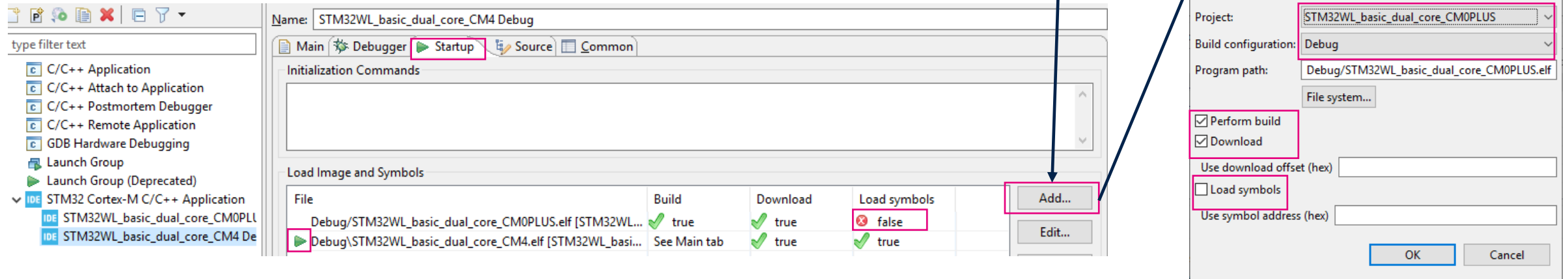
CortexM4 core



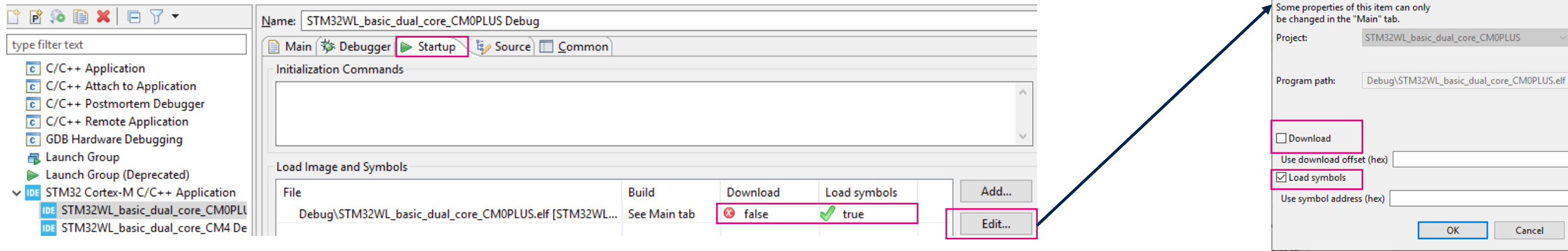
CortexM0+ core

STM32WL55 debug configuration how to configure startup debug for dual core

- Within debug startup configuration for CortexM4 it is necessary to add CortexM0+ part to be build and flashed together



- Within debug startup configuration for CortexM0+ only symbols for debug session need to be loaded (as code is already loaded with CM4 debug start)

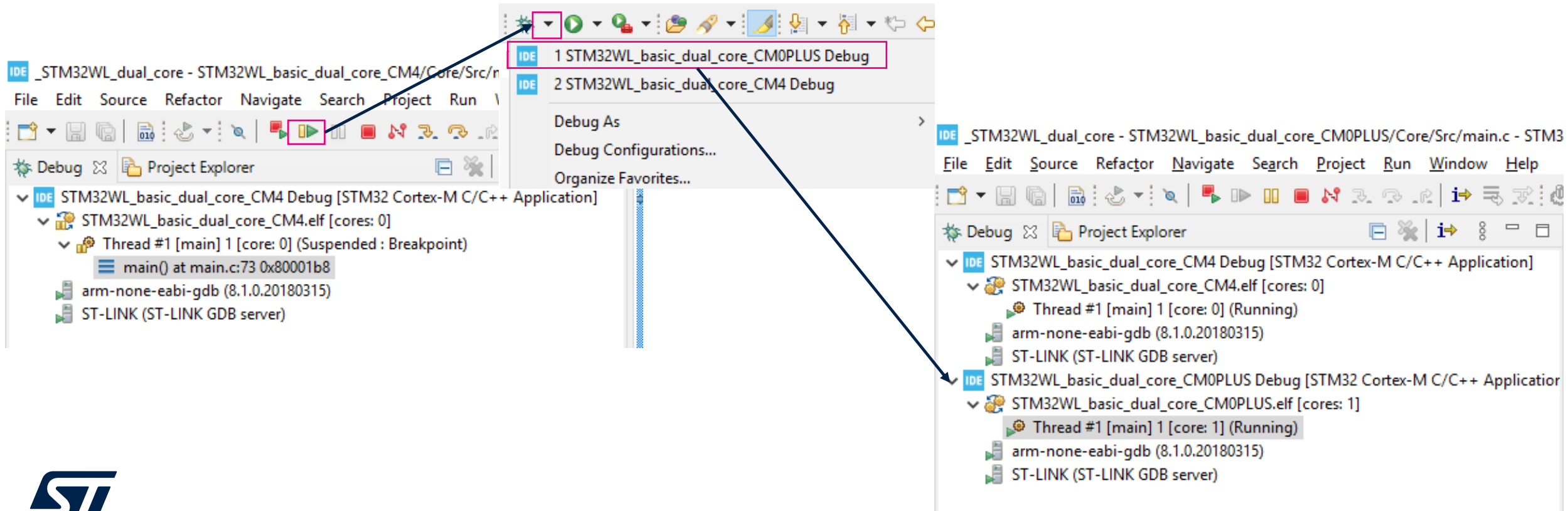


Dual core project debugging



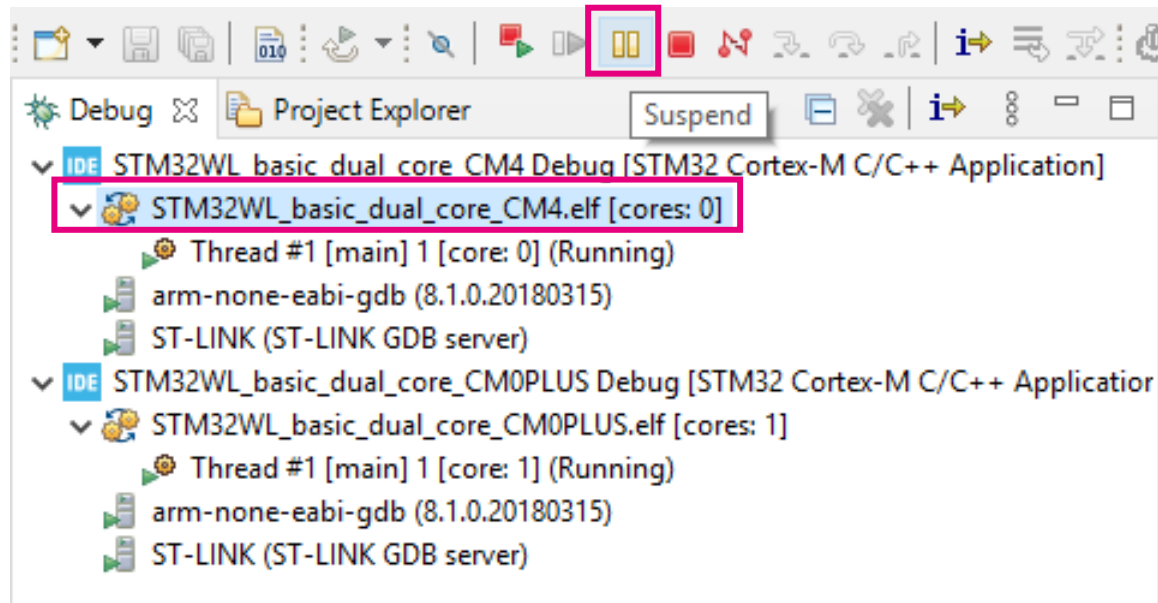
STM32WL55 debug configuration how to start debug session for dual core

- Start debug session for CortexM4 core
- Start code execution within debug perspective (to start CortexM0+ core)
- Start debug session for CortexM0+ core (it will load only symbols)

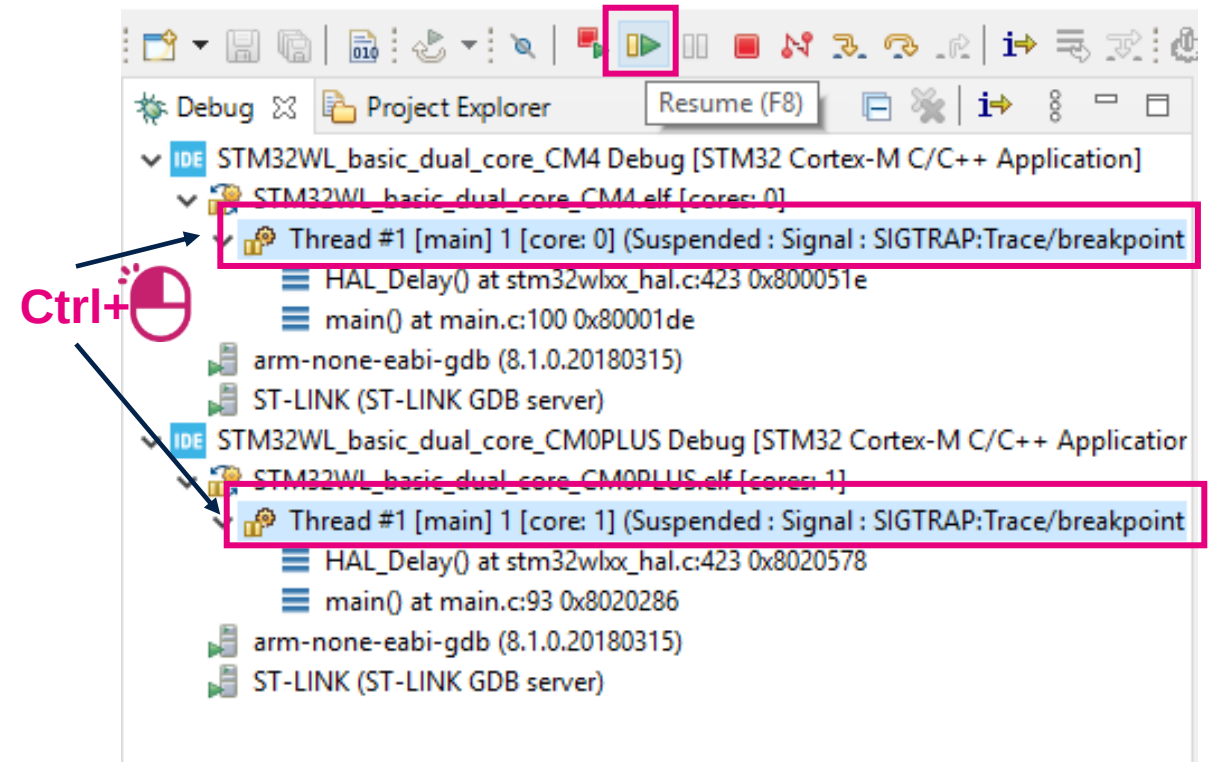


Synchronous pause and resume execution on both cores

- To suspend both cores stop at breakpoint or use Suspend button of any of the cores



- To resume both cores from suspend mode select both cores (threads) keeping Ctrl button pressed then press Resume button



Summary



Further information

- AN5564 - Getting started with projects based on dual-core STM32WL microcontrollers in STM32CubeIDE

Thank you

© STMicroelectronics - All rights reserved.

ST logo is a trademark or a registered trademark of STMicroelectronics International NV or its affiliates in the EU and/or other countries.

For additional information about ST trademarks, please refer to www.st.com/trademarks.

All other product or service names are the property of their respective owners.



life.augmented